XERIN GROUP COMPANY LTD

TECHNICAL ASSESSMENT

PRACTICAL DEVELOPER TEST

Delivery Management Backend System

Project	Delivery Management Backend System
Duration	3 Hours
Framework	NestJS
Language	TypeScript
Scope	Backend API Only

1.0 INTRODUCTION

1.1 A delivery company requires a backend system to manage customer orders, sellers, drivers and deliveries.

1.2 The company has the following resources:

Item	Quantity
Customer	1
Sellers	2 (Shop A and Shop B)
Delivery drivers	2
Products	10 (5 products per seller)

1.3 A customer purchases four products in a single checkout: two from Shop A and two from Shop B.

1.4 The system must create **one main order**, and split the delivery into **two independent shipments** (one per seller). Each shipment is assigned to a delivery driver.

2.0 REQUIRED SCENARIOS

No.	Scenario	Requirement
1	**Order Creation**	One purchase from both sellers creates one main order, with products organised into separate seller-based shipments.
2	**Driver Assignment**	Administrator assigns Shipment A to Driver 1 and Shipment B to Driver 2. Each driver accesses and manages only their own deliveries.

No.	Scenario	Requirement
3	**Independent Progress**	Driver 1 sets Shipment A to In Transit while Shipment B is not yet picked up. Each shipment keeps its own status without wrongly affecting the main order.
4	**Partial Delivery**	Shipment A is delivered; Shipment B is still processing. Customer sees one order with two shipment statuses. The order is not marked delivered.
5	**Failed Delivery**	Driver 2 cannot reach the customer and records a failed delivery with a reason. Delivery history is preserved and the appropriate next action is allowed.
6	**Delivery Completion**	Driver 2 delivers Shipment B. The system detects that all shipments are delivered and updates the overall order status.
7	**Delivery Tracking**	Customer or administrator retrieves full tracking: • Main order information • Shipments and assigned drivers • Individual shipment statuses • Progress history and completion information
8	**Security and Access Control**	Handle correctly: • Unauthenticated access to protected resources • Driver updating another driver's shipment • Driver accessing administrative operations • Assigning a shipment that is already delivered
9	**Data Integrity**	Maintain consistent data when: • Same checkout is submitted twice • Shipment creation fails halfway • Two drivers accept the same shipment • Invalid status update is submitted • A completed shipment receives another update

3.0 FINAL DELIVERABLE

3.1 Build the complete backend system using NestJS, functional and testable through Swagger UI.

3.2 Provide the **source code**, **database implementation** and **Swagger API documentation**.

3.3 No frontend, mobile application or UI development is required.

4.0 SYSTEM FLOWCHARTS

Figure 1: Order and Shipment Delivery Flow

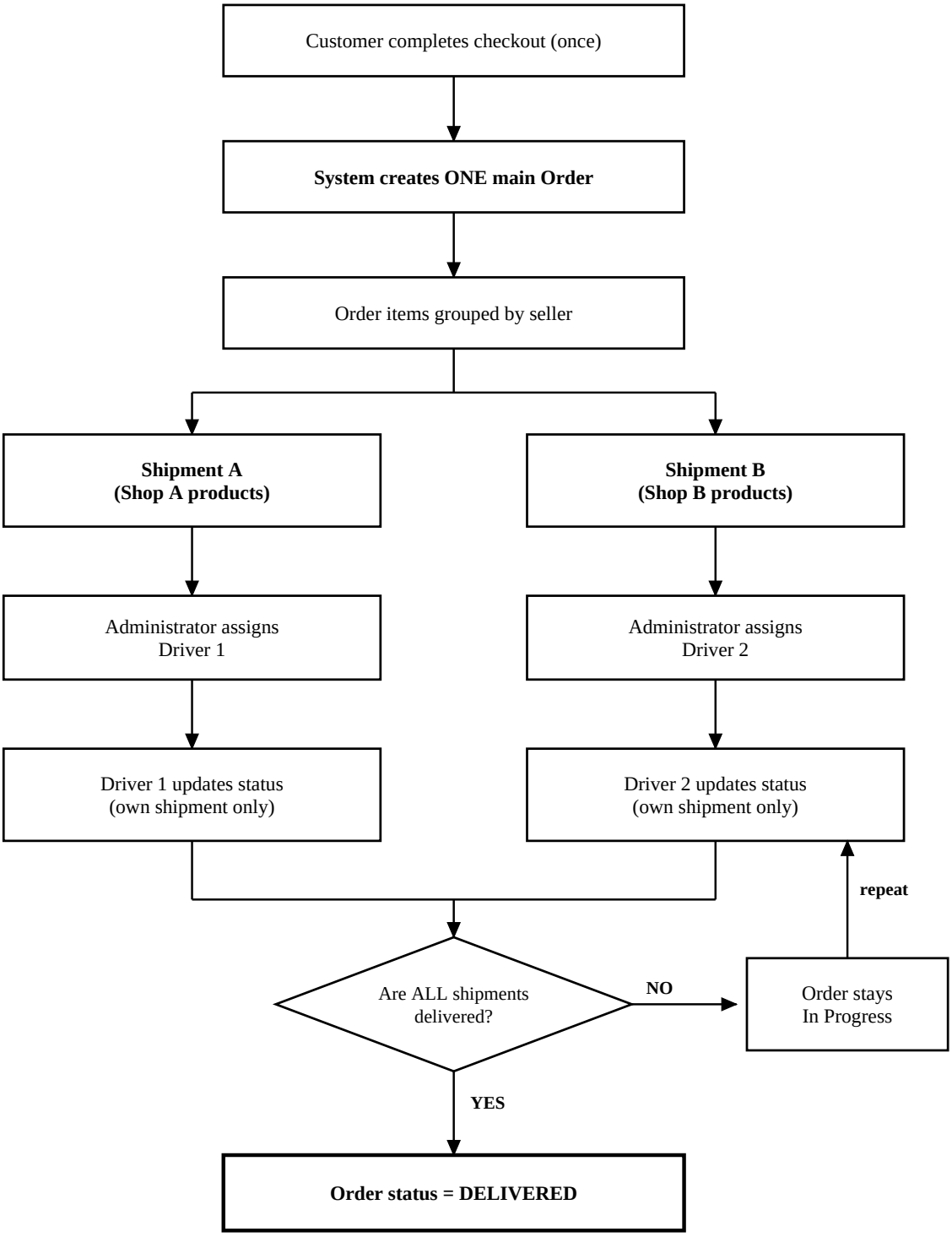
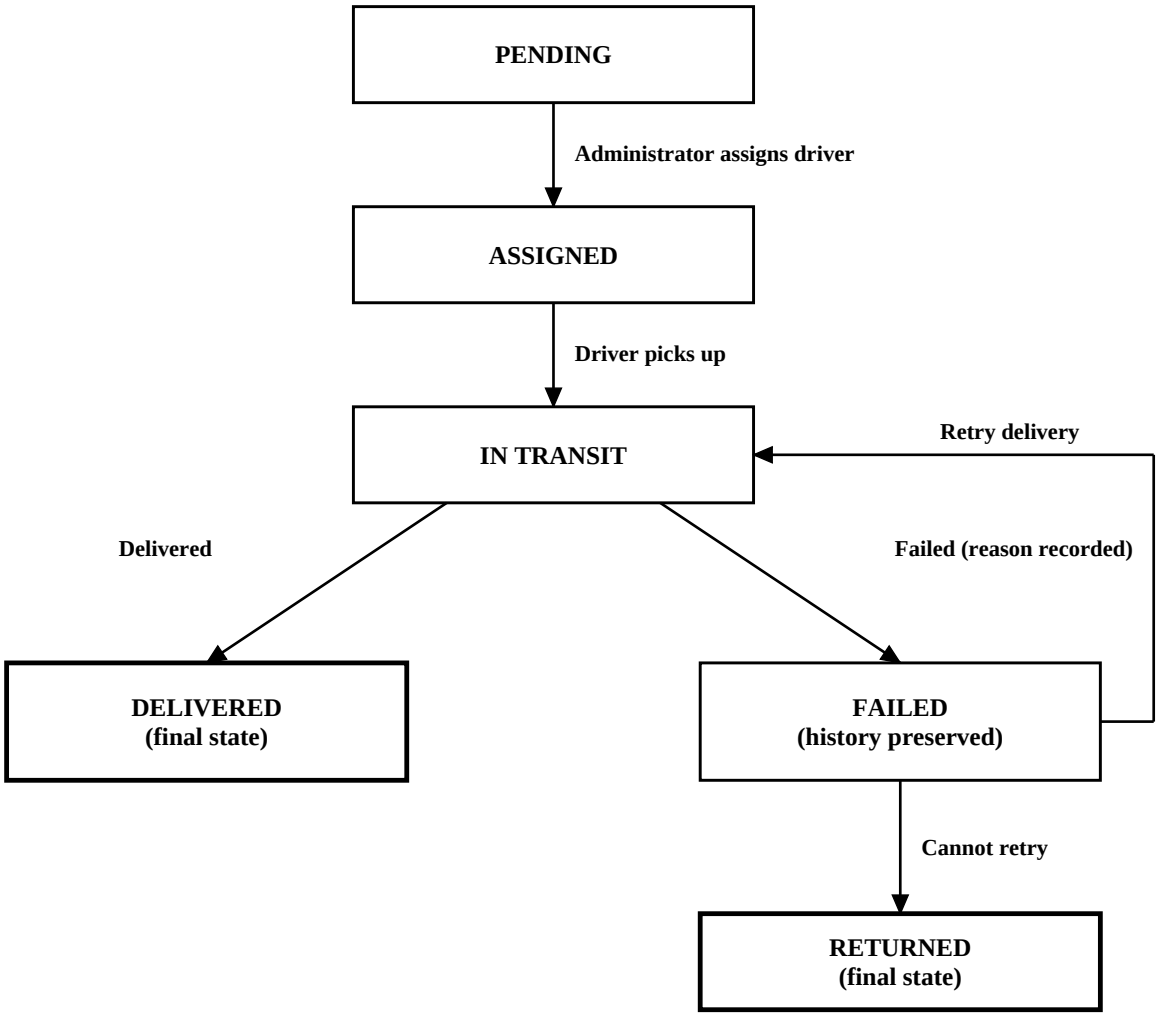


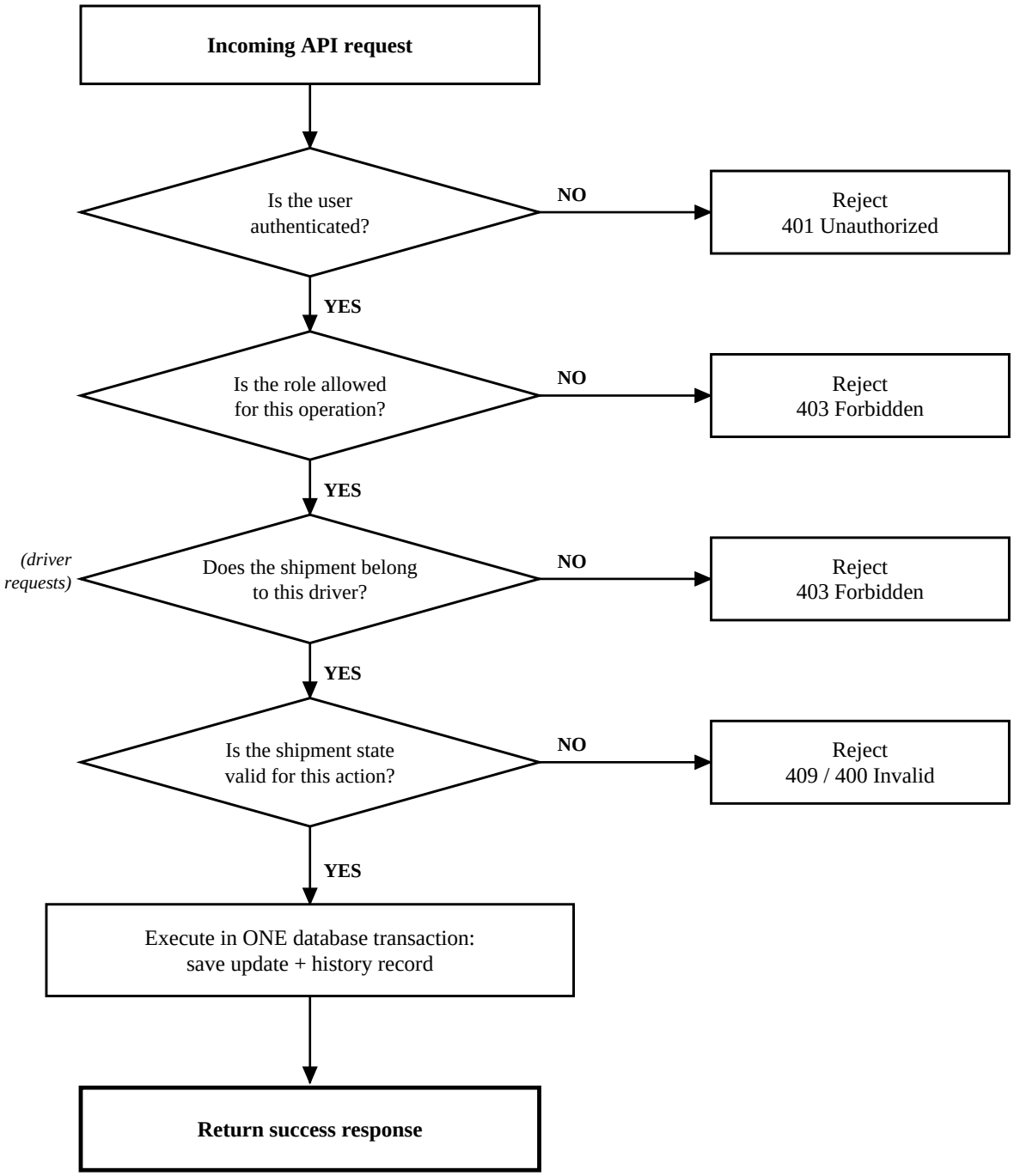
Figure 2: Shipment Status Flow



Final states cannot receive any further status update.

Status names are illustrative; candidates may adopt their own design.

Figure 3: Request Validation Flow (Security and Data Integrity)



Response codes are illustrative; candidates may adopt their own design.